



TSwap Protocol Audit Report

Version 1.0

web3pavlou

May 15, 2026

TSwap Protocol Audit Report

web3pavlou

May 15, 2026

Prepared by: web3pavlou

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price.

Disclaimer

The web3pavlou team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
1 Commit Hash: 1ec3c30253423eb4199827f59cf564cc575b46db
2 In Scope:
3 ./src/
4 #-- PoolFactory.sol
5 #-- TSwapPool.sol
6 Solc Version: 0.8.20
7 Chain(s) to deploy contract to: Ethereum
```

Roles

- **Liquidity Providers:** Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.

- **Users:** Users who want to swap tokens.

Executive Summary

This is an educational audit review from cyfrin-updraft educational material with some additions from my end. I spent some time writing the proof of code for each high severity vulnerability plus any other finding I spotted. There are probably more bugs to be found in this protocol.

The audit revealed a total of 12 findings across all severity levels. The most critical issues center around a broken core invariant, incorrect fee calculations, and missing slippage protection — all of which can result in direct loss of funds for users and liquidity providers.

Issues found

Severity	Number of Issues Found
High	5
Medium	1
Low	2
Informational	7
Total	15

Findings

High

[H-1] In TSwapPool : : _swap the extra tokens given to users after every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follows a strict invariant of $x * y = k$, Where: - x the balance of the pool Token - y the balance of WETH - k the constant product of the two balances

This means that whenever the balances change in the protocol, the ratio between the two token amounts remain constant. However this is broken due to the extra incentive in the `_swap` function. Meaning that over time the protocol funds will be drained.

The following block of code is responsible for this issue

```
1      swap_count++;
2      if (swap_count >= SWAP_COUNT_MAX) {
3          swap_count = 0;
4          outputToken.safeTransfer(msg.sender, 1
5                                   _000_000_000_000_000_000);
6      }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put the protocol's core invariant is broken.

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000
2. That user continues to swap until all the protocol's funds are drained

Code

Place the following into test/unit/TSwapPool.t.sol

```
1      function test_invariantBroken() public {
2          vm.startPrank(liquidityProvider);
3          weth.approve(address(pool), 100e18);
4          poolToken.approve(address(pool), 100e18);
5          pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6          vm.stopPrank();
7
8          uint256 outputWeth = 1e17;
9
10         vm.startPrank(user);
11         poolToken.approve(address(pool), type(uint256).max);
12         poolToken.mint(user, 100e18);
13         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
14         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
15         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
16         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
17         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
18         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
19         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
20     }
```

```
20     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
21         timestamp));  
22     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
23         timestamp));  
24     int256 startingY = int256(weth.balanceOf(address(pool)));  
25     int256 expectedDeltaY = int256(-1) * int256(outputWeth);  
26     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
27         timestamp));  
28     vm.stopPrank();  
29     uint256 endingY = weth.balanceOf(address(pool));  
30     int256 actualDeltaY = int256(endingY) - int256(startingY);  
31     assertEq(actualDeltaY, expectedDeltaY);  
32 }  
33
```

Recommended Mitigation: Remove the extra incentive mechanism, if you want to keep this in we should account for the change in the $x * y = k$ protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
1 -     swap_count++;  
2 -     if (swap_count >= SWAP_COUNT_MAX) {  
3 -         swap_count = 0;  
4 -         outputToken.safeTransfer(msg.sender, 1  
5 -             _000_000_000_000_000_000);  
6 -     }
```

[H-2] Incorrect fee calculation in TSwapPool::getInputAmountBasedOnOutput causes protocol to take too many tokens from users resulting in lost fees

Description: The `TSwapPool::getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens the user should deposit given an amount of output tokens. However the function currently miscalculates the resulting amount (fee precision). When calculating the fee, it scales the amount by 10_000 instead of 1_000.

```
1 function getInputAmountBasedOnOutput(  
2     uint256 outputAmount,  
3     uint256 inputReserves,  
4     uint256 outputReserves  
5 )  
6 public  
7 pure  
8 revertIfZero(outputAmount)  
9 revertIfZero(outputReserves)  
10 returns (uint256 inputAmount)
```

```
11     {
12     @>     return (inputReserves * outputAmount * 10000) / ((
13             outputReserves - outputAmount) * 997);
14     }
```

As a result, users swapping tokens via the `TSwapPool::swapExactOutput` function will pay far more tokens than expected for their trades. This becomes particularly risky for users that provide infinite allowance to the `TSwapPool` contract. Moreover, note that the issue is worsened by the fact that the `TSwapPool::swapExactOutput` function does not allow users to specify a maximum of input tokens, as is described in another issue in this report.

It's worth noting that the tokens paid by users are not lost, but rather can be swiftly taken by liquidity providers. Therefore, this contract could be used to trick users, have them swap their funds at unfavorable rates and finally rug pull all liquidity from the pool.

Impact: Protocol takes more fees than expected from users.

Proof of Concept:

Code

Place this in test/unit/TSwapPool.t.sol

```
1     function test_flawedExactOutput() public {
2         uint256 initialLiquidity = 100e18;
3         vm.startPrank(liquidityProvider);
4         weth.approve(address(pool), initialLiquidity);
5         poolToken.approve(address(pool), initialLiquidity);
6         pool.deposit({
7             wethToDeposit: initialLiquidity,
8             minimumLiquidityTokensToMint: 0,
9             maximumPoolTokensToDeposit: initialLiquidity,
10            deadline: uint64(block.timestamp)
11        });
12        vm.stopPrank();
13
14        address randomUser = makeAddr("randomUser");
15        uint256 userInitialBalance = 11e18;
16        poolToken.mint(randomUser, userInitialBalance);
17
18        vm.startPrank(randomUser);
19        poolToken.approve(address(pool), type(uint256).max);
20        pool.swapExactOutput(poolToken, weth, 1 ether, uint64(block.
21            timestamp));
22        vm.stopPrank();
23        // Initial liquidity was 1:1, so user should have paid ~1 pool
24        // token
25        // However, he spent much more than that. The user started with
26        // 11 tokens, and now only has less than 1.
```

```
24     assertLt(poolToken.balanceOf(randomUser), 1 ether);
25
26     // The liquidity provider can rug all funds from the pool now,
27     // including those deposited by user.
28     vm.startPrank(liquidityProvider);
29     pool.withdraw(
30         pool.balanceOf(liquidityProvider),
31         1, // minWethToWithdraw
32         1, // minPoolTokensToWithdraw
33         uint64(block.timestamp)
34     );
35     assertEq(weth.balanceOf(address(pool)), 0);
36     assertEq(poolToken.balanceOf(address(pool)), 0);
37 }
```

Recommended Mitigation:

```
1
2     function getInputAmountBasedOnOutput(
3         uint256 outputAmount,
4         uint256 inputReserves,
5         uint256 outputReserves
6     )
7     public
8     pure
9     revertIfZero(outputAmount)
10    revertIfZero(outputReserves)
11    returns (uint256 inputAmount)
12    {
13
14 -        return ((inputReserves * outputAmount) * 10000) / ((
15 +        return ((inputReserves * outputAmount) * 1000) / ((
16         outputReserves - outputAmount) * 997);
17         outputReserves - outputAmount) * 997);
18     }
```

[H-3] Lack of slippage protection in TSwapPool::swapExactOutput function, causes users to potentially receive way fewer tokens

Description: The `swapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput` where the function specifies `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept: 1. The pool starts with a 1:1 ratio — 100 poolToken : 100 WETH, so the user expects

to pay ~1 poolToken per 1 WETH output. 2. User submits a swapExactOutput looking for 1 WETH: - inputToken = poolToken - outputToken = WETH - outputAmount = 1e18 - deadline = whatever 3. The function offers no `maxInputAmount` parameter to protect the user. 4. While the tx is pending in the mempool, an attacker front-runs it — selling 80 poolToken into the pool via `swapExactInput`, draining ~44 WETH and heavily skewing the reserves to ~180 poolToken : ~55 WETH. 5. The user's tx executes in the now-skewed pool. To receive 1 WETH, the AMM now requires ~33 poolToken instead of the expected ~1 poolToken. 6. The transaction completes, and the user paid ~33x more than anticipated with no way to prevent it.

Code

Pasted this is test/unit/TSwapPool.t.sol

```
1     function test_slippageProtection() public {
2         // Setup initial liquidity 1:1 ratio
3         vm.startPrank(liquidityProvider);
4         weth.approve(address(pool), type(uint256).max);
5         poolToken.approve(address(pool), type(uint256).max);
6         pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
7         vm.stopPrank();
8
9         console.log("weth reserves before the attack: ", weth.balanceOf(
10             address(pool)));
11
12         // Attacker front-runs, skewing the pool
13         address attacker = makeAddr("attacker");
14         poolToken.mint(attacker, 80e18);
15         vm.startPrank(attacker);
16         poolToken.approve(address(pool), type(uint256).max);
17         pool.swapExactInput(poolToken, 80e18, weth, 0, uint64(block.
18             timestamp));
19         vm.stopPrank();
20
21         console.log("weth reserves after attack: ", weth.balanceOf(
22             address(pool)));
23         console.log("pooltoken reserves after attack:", poolToken.
24             balanceOf(address(pool)));
25
26         // User had checked the price BEFORE the attack — expected ~1
27         // e18 poolToken per 1e18 weth
28         uint256 expectedInputAmount = 1e18;
29
30         // Give user enough tokens to complete the now-expensive swap
31         poolToken.mint(user, 100e18);
32
33         vm.startPrank(user);
34         poolToken.approve(address(pool), type(uint256).max);
35         uint256 userPoolTokenBalanceBefore = poolToken.balanceOf(user);
```

```
32     console.log("User poolToken balance before swap:",
33                 userPoolTokenBalanceBefore);
34     // User asks for 1e18 weth – no `maxInputAmount`, so they can't
35     // cap what they spend
36     pool.swapExactOutput(poolToken, weth, 1e18, uint64(block.
37         timestamp));
38     uint256 actualInputAmount = userPoolTokenBalanceBefore -
39         poolToken.balanceOf(user);
40     vm.stopPrank();
41     console.log("Expected poolToken input (~1:1): ",
42                 expectedInputAmount);
43     console.log("Actual poolToken input paid:      ",
44                 actualInputAmount);
45     console.log("Overpaid by (in poolToken):      ",
46                 actualInputAmount - expectedInputAmount);
47     assert(actualInputAmount > expectedInputAmount);
48 }
```

Recommended Mitigation: We should include a `maxInputAmount` so the user only has to spend up to a specific amount and can predict how much they will spend on the protocol.

```
1  function swapExactOutput(
2      IERC20 inputToken,
3      + uint256 maxInputAmount
4      .
5      .
6      .
7      inputAmount = getInputAmountBasedOnOutput(outputAmount,
8          inputReserves, outputReserves);
9      + if(inputAmount > maxInputAmount){
10     +     revert;
11     + }
12     _swap(inputToken, inputAmount, outputToken, outputAmount);
13     .
14     .
15     .
```

[H-4] TSwapPool::sellPoolTokens mismatches input and output tokens causing users to receive incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive weth in exchange. Users indicate how many pool tokens they are willing to sell in

the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact that the `swapExactOutput` function is called whereas the `swapExactInput` function is the one that should be called because users specify the exact amount of input tokens not output.

Impact: Users will swap the wrong amount of tokens which is a severe disruption of protocol functionality

Proof of Concept:

Code

Place this in `test/unit/TSwapPool.t.sol`

```
1      function test_mismatchTokens() public {
2          // Setup initial liquidity 1:1 ratio
3          vm.startPrank(liquidityProvider);
4          weth.approve(address(pool), type(uint256).max);
5          poolToken.approve(address(pool), type(uint256).max);
6          pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
7          vm.stopPrank();
8
9          uint256 expectedPoolTokensSold = 50e18;
10         uint256 expectedWethReceived = 50e18;
11
12         vm.startPrank(user);
13         poolToken.mint(user, 1100e18);
14         poolToken.approve(address(pool), type(uint256).max);
15
16         uint256 userPoolTokensBefore = poolToken.balanceOf(user);
17         uint256 userWethBefore = weth.balanceOf(user);
18
19         pool.sellPoolTokens(50e18);
20
21         uint256 poolTokensSold = userPoolTokensBefore - poolToken.
22             balanceOf(user);
23         uint256 wethReceived = weth.balanceOf(user) - userWethBefore;
24         vm.stopPrank();
25
26         console.log("Expected poolTokens sold: ",
27             expectedPoolTokensSold);
28         console.log("Actual poolTokens sold:   ", poolTokensSold); //
29             ~1003e18 !!
30         console.log("Expected weth received:   ", expectedWethReceived)
31             ;
32         console.log("Actual weth received:     ", wethReceived);
33
34         assertLt(expectedPoolTokensSold, poolTokensSold);
35     }
```

```
32     }
```

Recommended Mitigation: Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolTokens` function to accept a new parameter (i.e. `minWethToReceive` to be passed to `swapExactInput`)

```
1     function sellPoolTokens(uint256 poolTokenAmount,
2 +   uint256 minWethToReceive
3   )
4     external returns (uint256 wethAmount) {
5 -     return swapExactOutput(i_poolToken, i_wethToken,
6 +     poolTokenAmount, uint64(block.timestamp));
7   }
6 +     return swapExactInput(i_poolToken, poolTokenAmount,
7   i_wethToken, minWethToReceive, uint64(block.timestamp));
8 }
```

Additionally, it might be wise to add a deadline to the function, as there is currently no deadline.

[H-5] TSwapPool::deposit pass uint64 deadline as an input parameter but never uses it, causing transaction to complete even after the deadline.

Description: The `deposit` function accepts a deadline parameter which according to the documentation is “The deadline for the transaction to be completed by”. However this parameter is never used and as a consequence operations that add liquidity to the pool may be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept: 1. A user calls `deposit` in a pool of tokens 2. He sets a deadline for his transaction to be eligible to pass 3. Time passes but the transaction still passes

Code

Place this in `test/unit/TSwapPool.t.sol`

```
1     function test_depositDeadlineNotEnforced() public {
2         vm.startPrank(liquidityProvider);
3         weth.approve(address(pool), type(uint256).max);
4         poolToken.approve(address(pool), type(uint256).max);
5
6         // User sets deadline for "right now"
7         uint64 deadline = uint64(block.timestamp);
8
9         // Tx sits in mempool - time passes (e.g. 1 hour)
10        vm.warp(block.timestamp + 3600);
```

```
11
12     // Deadline has now passed, but deposit still succeeds
13     pool.deposit(100e18, 0, 100e18, deadline);
14     vm.stopPrank();
15
16     assertEq(pool.totalLiquidityTokenSupply(), 100e18);
17 }
```

Recommended Mitigation: Consider making the following change to the function:

```
1
2 function deposit(
3     uint256 wethToDeposit,
4     uint256 minimumLiquidityTokensToMint,
5     uint256 maximumPoolTokensToDeposit,
6     uint64 deadline
7 )
8     external
9 +     revertIfDeadlinePassed(uint64 deadline)
10    revertIfZero(wethToDeposit)
11    returns (uint256 liquidityTokensToMint)
12 {}
```

Medium

[M-1] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant

Description: The protocol follows a strict invariant of $x * y = k$, where: - x is the balance of the pool token - y is the balance of WETH - k is the constant product of the two balances

However, certain ERC20 tokens behave unexpectedly and silently violate this invariant. Fee-on-transfer tokens (e.g. STA, PAXG) deliver less than the stated transfer amount, so the pool's accounting diverges from its actual balances on every interaction. Rebase tokens (e.g. stETH, AMPL) change holder balances outside of transfers, causing reserves to drift without any swap occurring. ERC777 tokens allow reentrant calls on transfer, enabling reentrancy attacks against the pool. None of these behaviors are guarded against in a scenario where a weird erc20 token pair-pool enables external calls like this.

Impact: The $x * y = k$ invariant is permanently broken from the first deposit. With fee-on-transfer tokens, every swap sends out more output than the pool actually received in input, slowly draining reserves. Over time liquidity providers withdraw less than they deposited, and the pool becomes insolvent.

Proof of Concept:

1. A fee-on-transfer token (10% fee) is used as the pool token.

2. A liquidity provider deposits 100 feeToken + 100 WETH. Due to the transfer fee, the pool only receives 90 feeToken — k is immediately $90 * 100 = 9000$ instead of the expected $100 * 100 = 10000$.
3. A user sells 10 feeToken via `swapExactInput`. The pool prices the WETH output assuming 10 feeToken arrive, but only 9 actually do.
4. The pool sends out WETH calculated on 10, receives only 9 — k shrinks further after every swap.
5. The invariant is permanently broken and the pool slowly drains.

Code

Add this mock to `test/mocks/FeeOnTransferTokenMock.sol`

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.20;
3
4 import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5
6 contract FeeOnTransferTokenMock is ERC20 {
7     uint256 public constant FEE_PERCENT = 10;
8
9     constructor() ERC20("FeeToken", "FEE") {}
10
11     function mint(address to, uint256 amount) external {
12         _mint(to, amount);
13     }
14
15     function _update(address from, address to, uint256 amount) internal
16         override {
17         if (from == address(0) || to == address(0)) {
18             super._update(from, to, amount);
19             return;
20         }
21         uint256 fee = (amount * FEE_PERCENT) / 100;
22         uint256 amountAfterFee = amount - fee;
23         super._update(from, address(0), fee); // burn the fee
24         super._update(from, to, amountAfterFee);
25     }
26 }
```

Place the following into `test/unit/TSwapPool.t.sol`:

```
1 function test_feeOnTransferTokenBreaksCoreInvariant() public {
2     FeeOnTransferTokenMock feeToken = new FeeOnTransferTokenMock();
3     TSwapPool feePool = new TSwapPool(address(feeToken), address(weth),
4         "LTokenB", "LB");
5
6     feeToken.mint(LiquidityProvider, 200e18);
7     weth.mint(LiquidityProvider, 200e18);
8 }
```

```
8     vm.startPrank(liquidityProvider);
9     weth.approve(address(feePool), type(uint256).max);
10    feeToken.approve(address(feePool), type(uint256).max);
11    feePool.deposit(100e18, 0, 100e18, uint64(block.timestamp));
12    vm.stopPrank();
13
14    uint256 wethReservesAfterDeposit = weth.balanceOf(address(feePool))
15    ;
16    uint256 feeTokenReservesAfterDeposit = feeToken.balanceOf(address(
17    feePool));
18    uint256 kBefore = wethReservesAfterDeposit *
19    feeTokenReservesAfterDeposit;
20
21    console.log("WETH intended to deposit:", uint256(100));
22    console.log("feeToken intended to deposit:", uint256(100));
23    console.log("WETH actually in pool:", wethReservesAfterDeposit / 1
24    e18); // 100
25    console.log("feeToken actually in pool:",
26    feeTokenReservesAfterDeposit / 1e18); // 90 !!
27    console.log("k after deposit (scaled):", kBefore / 1e36);
28    // 9000, not 10000
29
30    feeToken.mint(user, 100e18);
31    vm.startPrank(user);
32    feeToken.approve(address(feePool), type(uint256).max);
33
34    uint256 feeTokenReservesBefore = feeToken.balanceOf(address(feePool
35    ));
36    feePool.swapExactInput(feeToken, 10e18, weth, 0, uint64(block.
37    timestamp));
38    vm.stopPrank();
39
40    uint256 wethReservesAfter = weth.balanceOf(address(feePool));
41    uint256 feeTokenReservesAfter = feeToken.balanceOf(address(feePool
42    ));
43    uint256 kAfter = wethReservesAfter * feeTokenReservesAfter;
44
45    console.log("--- After swap ---");
46    console.log("feeToken user tried to sell: ", uint256(10));
47    console.log("feeToken pool actually got: ", (feeTokenReservesAfter
48    - feeTokenReservesBefore) / 1e18); // 9
49    console.log("k before swap (scaled): ", kBefore / 1e36);
50    console.log("k after swap (scaled): ", kAfter / 1e36); //
51    less than kBefore - invariant broken
52
53    assertNotEq(kBefore, kAfter);
54 }
```

Recommended Mitigation:

The most robust mitigations, in order of preference:

1. **Token whitelist** — maintain an allowlist of approved pool tokens vetted for standard ERC20 behavior. Reject pool creation for any token not on the list. This is what Uniswap V2 effectively does in practice.
2. **Balance checks around transfers** — rather than trusting the stated transfer amount, measure actual balance changes:

```
1 function _swap(...) {
2 +   uint256 balanceBefore = inputToken.balanceOf(address(this));
3   inputToken.safeTransferFrom(msg.sender, address(this), inputAmount)
4   ;
5 +   uint256 actualReceived = inputToken.balanceOf(address(this)) -
6   balanceBefore;
7 +   if (actualReceived < inputAmount) revert
8   TSwapPool__InsufficientInputReceived();
9   outputToken.safeTransfer(msg.sender, outputAmount);
10 }
```

3. **Documentation** — at minimum, explicitly document that fee-on-transfer, rebase, and ERC777 tokens are not supported, so integrators and users are warned.

For further reference on non-standard token behaviors you may visit this github repo: [weird-erc20](#)

Low

[L-1] TswapPool::_addLiquidityMintAndTransfer event has parameters out of order causing event to emit incorrect information

Description: When the `LiquidityAdded` event is emitted in the `TSwappPool::_addLiquidityMintAndTransfer` function it logs values in an incorrect order. The `poolTokenToDeposit` value should go in the third parameter position whereas the `wethToDeposit` value should be moved to the second.

Impact: Event emissions is incorrect, leading to off-chain functions malfunctioning

Recommended Mitigation:

```
1
2 -   emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit)
3   ;
4 +   emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit)
5   ;
```


[L-2] Default value returned by TSwapPool : : swapExactInput results in incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However while it declares the named return value `output` it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be zero, giving incorrect information to the caller.

Proof of Concept:

Code

In test/unit/TSwapPool.t.sol

```
1     function test_alwaysReturnIncorrectValue() public {
2         vm.startPrank(LiquidityProvider);
3         weth.approve(address(pool), 100e18);
4         poolToken.approve(address(pool), 100e18);
5         pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6         vm.stopPrank();
7
8         vm.startPrank(user);
9         weth.approve(address(pool), type(uint256).max);
10        uint256 output = pool.swapExactInput(weth, 1e18, poolToken, 0,
11            uint64(block.timestamp));
12        vm.stopPrank();
13        assertEq(output, 0);
14        assert(poolToken.balanceOf(user) > 10e18);
15    }
```

Recommended Mitigation:

```
1     {
2         uint256 inputReserves = inputToken.balanceOf(address(this));
3         uint256 outputReserves = outputToken.balanceOf(address(this));
4
5         -         uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount
6 +         , inputReserves, outputReserves);
7         +         output = getOutputAmountBasedOnInput(inputAmount, inputReserves
8 +         , outputReserves);
9         -         if (outputAmount < minOutputAmount) {
10        -             revert TSwapPool__OutputTooLow(outputAmount,
11        +             minOutputAmount);
12        +             if (output < minOutputAmount) {
13        +                 revert TSwapPool__OutputTooLow(output, minOutputAmount);
14        +             }
15        -         }
16        -         _swap(inputToken, inputAmount, outputToken, outputAmount);
```

```
15 +     _swap(inputToken, inputAmount, outputToken, output);
16 }
```

Informational

[I-1] Dead code should be removed.

Instances

- Found in src/PoolFactory.sol Line: 22
- Found in src/TSwapPool.sol Line: 131

```
1 -     error PoolFactory__PoolDoesNotExist(address tokenAddress);
2 -     uint256 poolTokenReserves = i_poolToken.balanceOf(address(this)
    );
```

[I-2] Lacking zero address check

Instances

```
1 constructor(
2     address poolToken,
3     address wethToken,
4     string memory liquidityTokenName,
5     string memory liquidityTokenSymbol
6 )
7     ERC20(liquidityTokenName, liquidityTokenSymbol)
8 {
9 +     if (wethToken == address(0) || poolToken == address(0)) {
10 +         revert TSwapPool__InvalidToken();
11 +     }
12
13     i_wethToken = IERC20(wethToken);
14     i_poolToken = IERC20(poolToken);
15 }
```

[I-3] PoolFactory::createPool should use .symbol() instead of .name.

In `PoolFactory::createPool` this bit `string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).name());` parses the same name parameter for the pool itself twice. We should instead pass the symbol argument and not the name.

Instances

```
1  function createPool(address tokenAddress) external returns (address) {
2      .
3      .
4      .
5  -      string memory liquidityTokenSymbol = string.concat("ts", IERC20
      (tokenAddress).name());
6  +      string memory liquidityTokenSymbol = string.concat("ts", IERC20
      (tokenAddress).symbol());
7      TSwapPool tPool = new TSwapPool(tokenAddress, i_wethToken,
      liquidityTokenName, liquidityTokenSymbol);
8      .
9      .
10     .
11 }
```

[I-4] TSwapPool events should be indexed

```
1  - event Swap(address indexed swapper, IERC20 tokenIn, uint256
    amountTokenIn, IERC20 tokenOut, uint256 amountTokenOut);
2  + event Swap(address indexed swapper, IERC20 indexed tokenIn, uint256
    amountTokenIn, IERC20 indexed tokenOut, uint256 amountTokenOut);
```

[I-5] TSwapPool::deposit does not fully follow the Checks-Effects-Interactions pattern

Inside the initial liquidity branch of `TSwapPool::deposit`, the return variable `liquidityTokensToMint` is assigned after the external interactions performed by `_addLiquidityMintAndTransfer`.

While this does not currently introduce a direct vulnerability since `liquidityTokensToMint` is only a local variable, following CEI consistently improves readability, reduces future maintenance risk, and helps prevent accidental introduction of state inconsistencies during future refactors.

```
1  } else {
2  +      liquidityTokensToMint = wethToDeposit;
3
4      _addLiquidityMintAndTransfer(
5          wethToDeposit,
6          maximumPoolTokensToDeposit,
7          wethToDeposit
8      );
9
10 -      liquidityTokensToMint = wethToDeposit;
11 }
```

[I-6] Usage of magic numbers is discouraged

The protocol uses hardcoded numeric literals throughout pricing calculations:

```
1 uint256 inputAmountMinusFee = inputAmount * 997;  
2 uint256 denominator = (inputReserves * 1000) + inputAmountMinusFee;  
3 return ((inputReserves * outputAmount) * 10000) / ((outputReserves -  
    outputAmount) * 997);
```

Using named constants improves readability, maintainability, and reduces the likelihood of inconsistencies during future fee updates.

This codebase already demonstrates the risks of hardcoded literals. In `TSwapPool::getInputAmountBasedOnOutputAmount`, the usage of 10000 instead of 1000 introduces incorrect fee calculations and results in users being charged significantly more input tokens than intended.

Consider replacing these literals with descriptive constants such as:

```
1 uint256 private constant FEE_PRECISION = 1000;  
2 uint256 private constant FEE_MULTIPLIER = 997;
```

[I-7] Public Function Not Used Internally

If a function is marked public but is not used internally, consider marking it as `external`.

1 Found Instances

- Found in `src/TSwapPool.sol` Line: 265

```
1     function swapExactInput(  
2         IERC20 inputToken,  
3         uint256 inputAmount,  
4         IERC20 outputToken,  
5         uint256 minOutputAmount,  
6         uint64 deadline  
7     )  
8  
9     public  
10    revertIfZero(inputAmount)  
11    revertIfDeadlinePassed(deadline)  
12  
13    returns (uint256 output){}
```